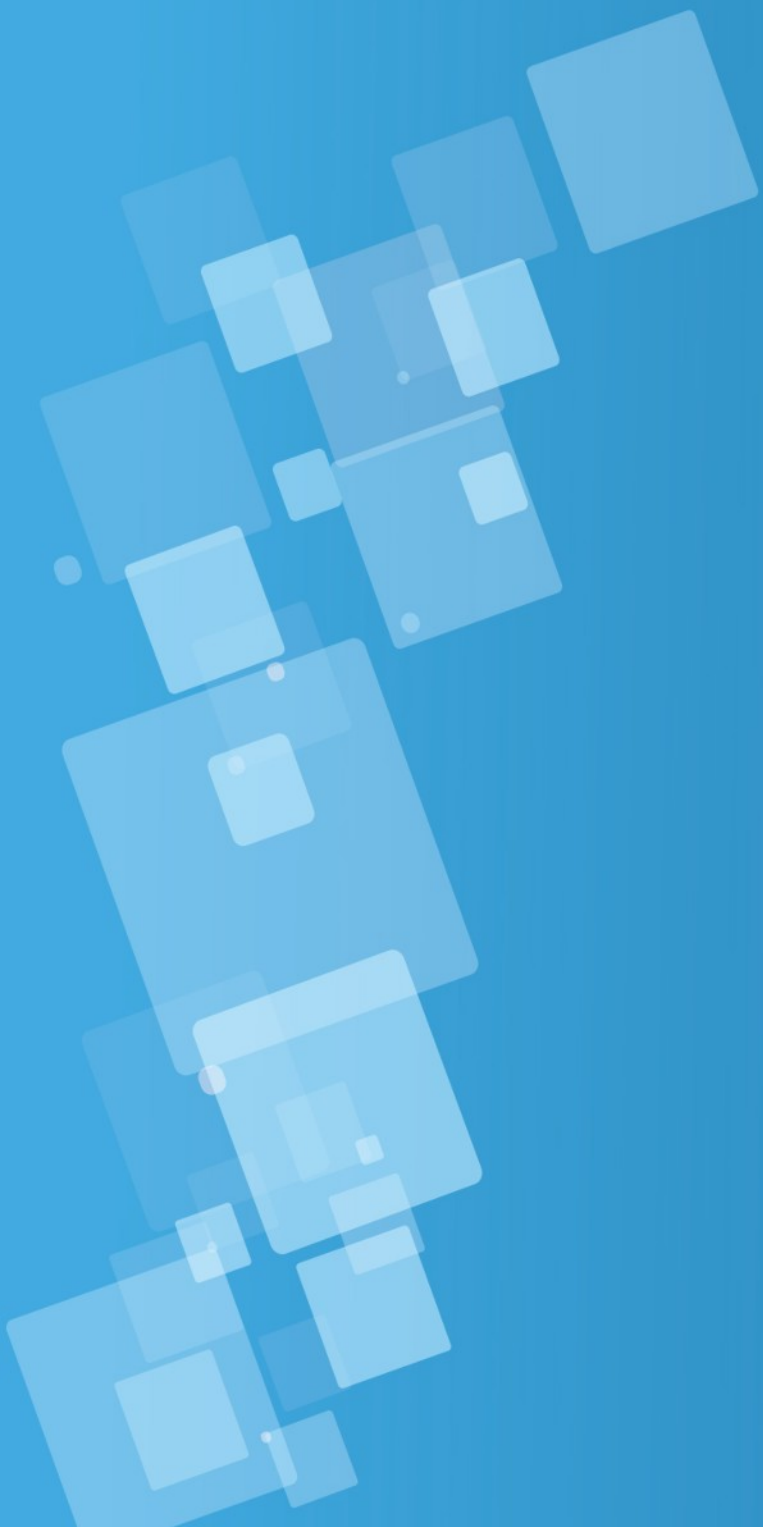




Técnicas Digitales II

RTOS (Real Time Operating System)
Sistema operativo en tiempo real
Gestión de interrupciones



Naturaleza y Detección de Eventos

¿ Qué es un evento ?

- Suceso que se origina en el entorno: (Llegada de un paquete de red, pulsación de un botón, etc).
- Requiere una acción por parte del sistema.

¿ Cómo detectamos un evento ?

- **Interrupciones:** método general y eficiente.
- **Sondeo (polling):** requiere una acción por parte del sistema.



Estrategia e Implementación

Como estrategia de implementación debemos resolver:

- Cuánto del procesamiento se realiza dentro de la Rutina de Servicio de Interrupción (ISR) y cuánto se realiza fuera de ella.
 - Se considera una buena práctica mantener las ISRs lo más cortas posible.
- Cómo comunicar el evento desde la ISR al código principal de la aplicación.

FreeRTOS no impone una estrategia única, sino que proporciona las características necesarias para implementar la estrategia elegida de forma simple y mantenible .



Tareas vs. Interrupciones

Prioridad de Tarea vs. Prioridad de Interrupción

- **Prioridad de Tarea:** Es un concepto de software gestionado por el planificador (scheduler) de FreeRTOS .
- **Prioridad de Interrupción:** Es un concepto de hardware gestionado por el procesador.

Importante: El hardware siempre tiene precedencia.

- La interrupción de más baja prioridad siempre interrumpirá la ejecución de la tarea de más alta prioridad.
- Una tarea nunca puede interrumpir (hacer "preemption") a una ISR.



El API Seguro para Interrupciones

- Muchas funciones de FreeRTOS (como vTaskDelay o xQueueReceive con bloqueo) no pueden ser llamadas desde una ISR. Esto es porque una ISR no se puede “bloquear”
- FreeRTOS proporciona versiones alternativas de sus funciones, seguras para interrupciones.
- Dentro de una ISR, solo puedes usar funciones que terminen en ...FromISR()

Importante: no se pueden llamar funciones de la API de FreeRTOS que no tengan “FromISR” en su nombre



¿Por qué un API Separado?

Ventajas:

- No necesitan lógica adicional para saber si se llaman desde una tarea o no
- Algunos parámetros cambian según si es desde tarea o desde ISR
- Cada port de FreeRTOS necesitaría proporcionar un mecanismo para determinar si es (tarea o ISR).
- Si no es fácil para la arquitectura determinar si está en tarea o ISR, necesitará código de entrada extra en la interrupción sumando complejidad e ineficiencia.



¿Por qué un API Separado?

Problema:

Si se llama a una función desde una tarea y desde una ISR, esta función no es API de FreeRTOS, pero llama a una API

Esto puede ocurrir con librería de terceros, en este caso la solución es:

- Se canaliza el procesamiento de una interrupción a una tarea (la API se llama solo desde una tarea)
- Si se puede, utilizar la función que termina en “FromISR”, esta puede llamarse desde una tarea (lo inverso no)
- Si el código de tercero incluye capa de abstracción de RTOS, se puede comprobar el contexto para saber si estamos en tarea o ISR (esto agrega overhead pero simplifica estos casos)



El Parámetro: pxHigherPriorityTaskWoken

Las funciones ...FromISR() tienen un parámetro especial, usualmente pxHigherPriorityTaskWoken.

La ISR usa este puntero para "decirle" al kernel si su acción (ej. dar un semáforo) ha despertado a una tarea de prioridad MÁS ALTA que la tarea que estaba ejecutándose.

Algunas API pueden mover una tarea del estado **Blocked** a **Ready**, funciones como xQueueSendToBack() pueden desbloquear una tarea que estaba esperando un dato.

Si la tarea desbloqueada es de mayor prioridad que la tarea en ejecución el scheduler selecciona a la tarea desbloqueada



El Parámetro: `pxHigherPriorityTaskWoken`

El momento del cambio depende del contexto

- **API desde una tarea**

Si `configUSE_PREEMPTION=1` se cambia la tarea inmediatamente, antes que la función API haya terminado.

- **API desde una ISR**

El cambio de tarea no se produce automáticamente, la variable `*pxHigherPriorityTaskWoken` devuelve `pdTRUE` notificando a la función sobre el requerimiento del cambio, es cuestión del desarrollador realizar las acciones o no, de no hacer nada en el próximo tick se realizará el cambio.

Importante: `pxHigherPriorityTaskWoken` se inicializa con `pdFALSE`, si hay mas de una API se pasa la misma variable en todas ellas, y al final de acuerdo al estado se toma la decisión de cambiar la tarea o no



¿Cómo usar pxHigherPriorityTaskWoken?

```
void vUnaISR(void) {
    BaseType_t xHigherPriorityTaskWoken;
    // Inicializar la variable a pdFALSE (siempre se debe inicializar)
    xHigherPriorityTaskWoken = pdFALSE;

    // Llamar al API de ISR y pasarle la dirección
    xSemaphoreGiveFromISR(xSemaphore, &xHigherPriorityTaskWoken);

    // Al final de la ISR, verificar si la variable se puso a pdTRUE
    if (xHigherPriorityTaskWoken == pdTRUE) {
        // Se despertó una tarea más importante.
        // Forzamos un cambio de contexto.
    }
}
```



Forzando un Cambio de Contexto

- Si xHigherPriorityTaskWoken es pdTRUE, debemos forzar un cambio de contexto para que la tarea de alta prioridad se ejecute inmediatamente al salir de la ISR.
- Esto se hace con los macros portYIELD_FROM_ISR() o portEND_SWITCHING_ISR().
- Ambas hacen lo mismo
- Se les pasa el parámetro xHigherPriorityTaskWoken, la macro solo solicita cambio de tarea si el parámetro es pdTRUE

```
void vUnaISR(void) {  
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;  
    xSemaphoreGiveFromISR(xSemaphore, &xHigherPriorityTaskWoken);  
    // Pasa la variable al macro.  
  
    portYIELD_FROM_ISR(xHigherPriorityTaskWoken);  
}
```



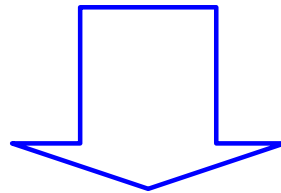
Procesamiento Diferido de Interrupciones

Se considera una buena práctica mantener lo (ISR) lo más breves posible. Las razones para ello son las siguientes:

- Las tareas solo se ejecutarán si el hardware no está atendiendo interrupciones.
- Una ISR pueden alterar (añadir “jitter”) tanto el tiempo de inicio como el de ejecución de una tarea.
- Es posible que la arquitectura no acepte nuevas interrupciones, o al menos un subconjunto de ellas, mientras se ejecuta una ISR.
- Se debe considerar que tarea y ISR accedan simultáneamente a al mismo recursos.
- El anidamiento de interrupciones puede aumentar la complejidad y reducir la predictibilidad. (mas corta la ISR menor probabilidad de que se anide)

Procesamiento Diferido de Interrupciones

- La ISR captura el evento, obtiene el dato (si es pequeño) y limpia la interrupción.
- Los demás procesamiento se realizan en una tarea
 - Permite priorizar el procesamiento con respecto a otras tareas de la aplicación.
 - En caso que la tarea a la que se difiere tiene la mayor prioridad, el procesamiento se realizará de inmediato, como si se hubiera realizado en la propia ISR.

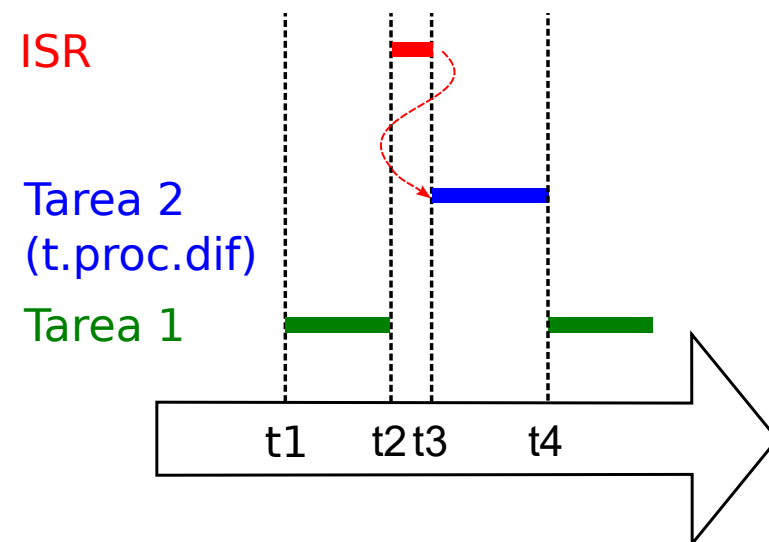


Procesamiento de Interrupción Diferido

Procesamiento Diferido de Interrupciones

- En t2 la tarea 1 se está ejecutando cuando ocurre una interrupción
- A partir de t2 se ejecuta la ISR donde atiende al periférico, borra la interrupción y desbloquea a la tarea 2
- La tarea 2 por ser la de mayor prioridad, comienza luego de que la ISR termina, completando el procedimiento de la interrupción
- En t4 finaliza la tarea 2 continuando la tarea 1

Si bien el procedimiento de la interrupción ocupa de t2 a t4, solo t2 a t3 se ejecuta en la ISR





Procesamiento Diferido de Interrupciones

Esto es principalmente útil cuando:

- El procesamiento requerido por la interrupción no es trivial. Se requiere pasar por un filtro de software por ejemplo
- Se requiere acciones que no pueden realizarse dentro de una ISR. Como escribir en la consola o asignar memoria.
- No es determinista; es decir, no se sabe de antemano cuánto tiempo tardará.



Uso de Semáforos Binarios para Sincronización

Semáforo binario:

- Permite desbloquear una tarea
- Se tiene la versión ISR para desbloquear tarea dentro de la interrupción.

Si el procedimiento es critico:

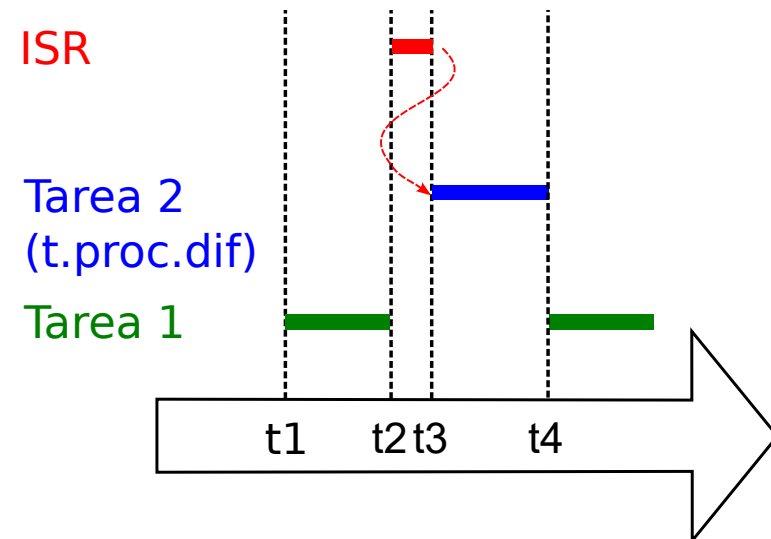
- Se establece la tarea con alta prioridad
- Usando `portYIELD_FROM_ISR()`, la tarea de procesamiento diferido arrancará inmediatamente.

Esto permite simular la ejecución como si hubiéramos implementado todo dentro de la ISR.

Semáforos Binarios para Sincronización

Procedimiento

- en t2 a t3 la ISR atiende al periférico, luego da una señal al semáforo para desbloquear la tarea 2
- La tarea 2 de mayor prioridad se desbloquea y completa el proceso, luego se bloquea esperando una nueva señal del semáforo





Semáforos Binarios para Sincronización

El Semáforo Binario, es entonces el mecanismo más simple para implementar el procesamiento diferido

Pensemos en un semáforo binario como una cola de longitud 1.

- La tarea "toma" (lee) el semáforo. Si está vacío, la tarea se bloquea.
- La ISR "da" (escribe) el semáforo, desbloqueando a la tarea.

El mecanismo es diferente de otros usos del Semáforo, por ejemplo una tarea toma el semáforo para utilizar un recurso y luego es la misma función quien lo devuelve



API: Creación de un Semáforo Binario

```
SemaphoreHandle_t xSemaphoreCreateBinary(void);
```

Crea un semáforo binario.

- Devuelve un SemaphoreHandle_t (un puntero) que se usará para referenciar el semáforo.
- Devuelve NULL si no hay suficiente memoria para crearlo

Un semáforo binario se crea "vacío" (no se puede tomar).



API: Tomar un Semáforo (en una Tarea)

```
BaseType_t xSemaphoreTake(SemaphoreHandle_t xSemaphore,  
                          TickType_t xTicksToWait);
```

Se usa para "tomar" o "recibir" un semáforo.

Es solo para tareas, **NUNCA SE DEBE USAR EN UNA ISR.**

- xTicksToWait: Tiempo máximo que la tarea se bloqueará si el semáforo no está disponible.
 - portMAX_DELAY significa esperar indefinidamente.
- Devuelve
 - pdPASS si se obtuvo con éxito
 - pdFALSE si pasado el tiempo de bloqueo, el semáforo aún no está disponible



API: Dar un Semáforo (en una ISR)

```
 BaseType_t xSemaphoreGiveFromISR( SemaphoreHandle_t xSemaphore,  
                                   BaseType_t *pxHigherPriorityTaskWoken );
```

Se usa para "dar" o "liberar" un semáforo desde una interrupción.

Es la versión para ISR de xSemaphoreGive()

- pxHigherPriorityTaskWoken: El puntero que vimos antes, para notificar al kernel si se requiere un cambio de contexto.
- Devuelve
 - pdPASS se ejecutó con éxito
 - pdFAIL sino se puede liberar

Diagrama de Flujo: Tarea + ISR + Semáforo

Sincronización con Semáforo Binario

1. La Tarea llama a `xSemaphoreTake()` y se bloquea (0% CPU).
2. Ocurre el evento de hardware (Interrupción).
3. La ISR se ejecuta y llama a `xSemaphoreGiveFromISR()`.
4. El semáforo se libera, la Tarea Lógica pasa a estado Ready.
5. La ISR termina. Si la Tarea Lógica es la de mayor prioridad, el scheduler la ejecuta.
6. La Tarea Lógica procesa el evento y vuelve al paso 1.

`xSemaphoreGiveFromISR()`

Dar al semáforo



Semáforo



`xSemaphoreTake()`

Recibir del semáforo



Ejemplo 16 - Sincronización ISR-Tarea

Objetivo: Sincronizar una tarea (vHandlerTask) con una interrupción generada por software (vPeriodicTask).

Componentes:

- vPeriodicTask (Baja Prioridad): Genera una interrupción cada 500ms.
- ulExampleInterruptHandler (ISR): Da un semáforo binario.
- vHandlerTask (Alta Prioridad): Se bloquea esperando el semáforo.

Ejemplo 16 - Generador ISR

Para simplificar el ejemplo, creamos una tarea que solo genera una interrupción por software cada 500 ms

```
/* esto muestra el port para windows que usa el libro */
#define mainINTERRUPT_NUMBER 3
static void vPeriodicTask( void *pvParameters ) {
    const TickType_t xDelay500ms = pdMS_TO_TICKS( 500UL );
    for( ;; ) {
        vTaskDelay( xDelay500ms );
        /* se imprime un mensaje antes y después de generar la
interrupción */
        vPrintString( "Periodic task - About to generate an interrupt.\r\n" );
        vPortGenerateSimulatedInterrupt( mainINTERRUPT_NUMBER );
        vPrintString( "Periodic task - Interrupt generated.\r\n\r\n\r\n" );
    }
}
```

Ejemplo 16 - Código de la ISR

Esta rutina atiende la interrupción, dentro de ella desbloquea al semáforo para que la tarea complete el procesamiento de la interrupción

```
static uint32_t ulExampleInterruptHandler( void ) {
    BaseType_t xHigherPriorityTaskWoken;
    /* se inicializa xHigherPriorityTaskWoken a pdFALSE */
    xHigherPriorityTaskWoken = pdFALSE;
    /* entrega de semáforo para desbloquear la tarea */
    xSemaphoreGiveFromISR( xBinarySemaphore, &xHigherPriorityTaskWoken );
    /* se pasa xHigherPriorityTaskWoken a portYIELD_FROM_ISR(). si
    xHigherPriorityTaskWoken es pdTRUE se debe hacer cambio de contexto */
    portYIELD_FROM_ISR( xHigherPriorityTaskWoken );
}
```



Ejemplo 16 - Código de la Tarea "Handler"

Tarea que completa la interrupción

```
static void vHandlerTask( void *pvParameters ) {  
    for( ;; ) {  
        /* utilizamos el semáforo para esperar el evento */  
        xSemaphoreTake( xBinarySemaphore, portMAX_DELAY );  
        /* cuando ocurre el evento, lo procesamos, en este caso solo  
imprimimos un mensaje */  
        vPrintString( "Handler task - Processing event.\r\n" );  
    }  
}
```

Ejemplo 16 - Código de la Tarea "Handler"

main

```
int main( void ) {
    /* crea el semáforo */
    xBinarySemaphore = xSemaphoreCreateBinary();
    if( xBinarySemaphore != NULL ) {
        /* tarea para manejar la interrupción */
        xTaskCreate( vHandlerTask, "Handler", 1000, NULL, 3, NULL );
        /* tarea para generar una interrupción cada 500ms */
        xTaskCreate( vPeriodicTask, "Periodic", 1000, NULL, 1, NULL );
        /* implementación de la interrupción por software (manejo exclusivo
para el port de windows */
        vPortSetInterruptHandler(mainINTERRUPT_NUMBER, ulExampleInterruptHandler);

        vTaskStartScheduler();
    }
    for( ;; );
}
```

Ejemplo 16 - Diagrama de Ejecución

La salida del vHandlerTask interrumpe la salida del vPeriodicTask. Esto demuestra que la ISR y el scheduler forzaron un cambio de contexto inmediato a la tarea de alta prioridad.

C:\WINDOWS\system32\cmd.exe - rtosdemo

Handler task - Processing event.
Periodic task - Interrupt generated.

Periodic task - About to generate an interrupt.
Handler task - Processing event.
Periodic task - Interrupt generated.

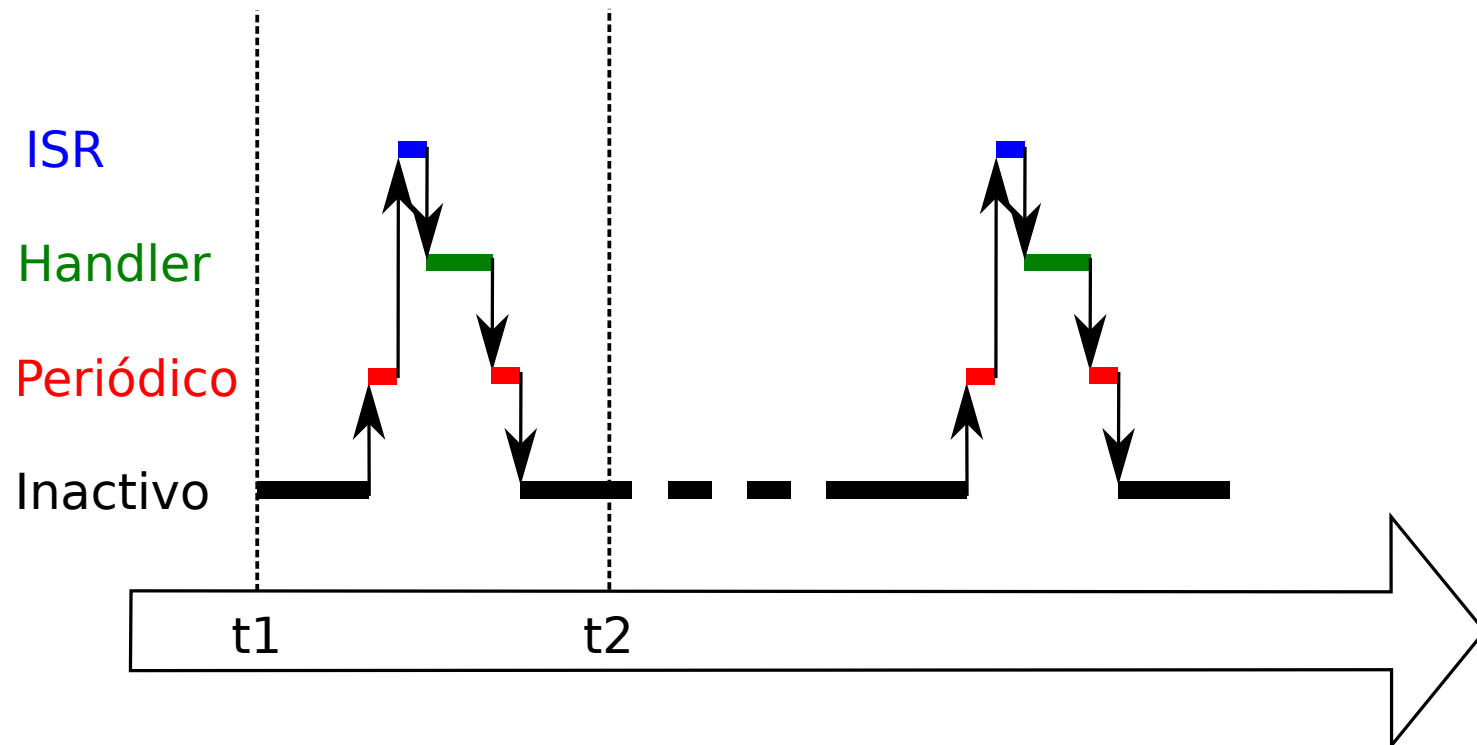
Periodic task - About to generate an interrupt.
Handler task - Processing event.
Periodic task - Interrupt generated.

Periodic task - About to generate an interrupt.
Handler task - Processing event.
Periodic task - Interrupt generated.

Periodic task - About to generate an interrupt.
Handler task - Processing event.
Periodic task - Interrupt generated.

Ejemplo 16 - Salida y Análisis

- vPeriodicTask (Periódico) es interrumpida por la ISR y esta se ejecuta
- la ISR retorna directamente a vHandlerTask (T-H), que se ejecuta antes de que la tarea periódica pueda terminar de imprimir su segundo mensaje.





¿ Preguntas ?